

Serial Communication (UART, SPI, and I2C)

Tina Tazehdel, Sangyun Kim

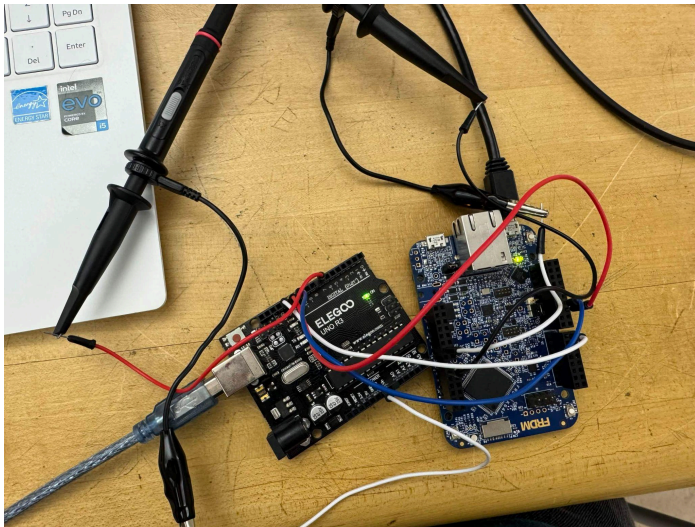
Objective

The objective of this lab was to gain experience using UART, SPI, and I2C with the FRDM-K64F microcontroller using Processor expert in kinetis design studio. In part 2 of the lab, SPI communication was established between the K64F board as the master device and an arduino UNO as the slave device.

Experiment system specifications

SPI Signal	Arduino Uno Pin	K64F Pin
MOSI	11	PTD2
MISO	12	PTD3
SCK	13	PTC5
SS	unused	unused

pics:



Software Design

```
/* #####
**  Filename   : main.c
**  Project    : Lab6_Part2
**  Processor  : MK64FN1M0VLL12
**  Version    : Driver 01.01
**  Compiler   : GNU C Compiler
**  Date/Time  : 2019-11-03, 17:50, # CodeGen: 0
**  Abstract   :
**      Main module.
**      This module contains user's application code.
**  Settings   :
**  Contents   :
**      No public methods
**
** #####*/
/*!
** @file main.c
** @version 01.01
** @brief
**      Main module.
**      This module contains user's application code.
*/
/*!
** @addtogroup main_module main module documentation
** @{
*/
/* MODULE main */

/* Including needed modules to compile this module/procedure */
#include "Cpu.h"
#include "Events.h"
#include "Pins1.h"
#include "FX1.h"
#include "GI2C1.h"
#include "WAIT1.h"
#include "CI2C1.h"
#include "CsIO1.h"
#include "IO1.h"
#include "MCUC1.h"
#include "SM1.h"
/* Including shared modules, which are used for whole project */
#include "PE_Types.h"
#include "PE_Error.h"
#include "PE_Const.h"
#include "IO_Map.h"
#include "PDD_Includes.h"
#include "Init_Config.h"
```

```

/* User includes (#include below this line is not maintained by Processor Expert) */
#include "MK64F12.h"
/*lint -save -e970 Disable MISRA rule (6.3) checking. */

unsigned char write[512];
int main(void)
/*lint -restore Enable MISRA rule (6.3) checking. */
{
    /* Write your local variable definition here */

    /** Processor Expert internal initialization. DON'T REMOVE THIS CODE!!! */
    PE_low_level_init();
    /** End of Processor Expert internal initialization. */
    /* Write your code here */
    uint32_t delay;
    uint8_t ret, who;
    int8_t temp;
    int16_t accX, accY, accZ;
    int16_t magX, magY, magZ;

    //initialization for string delay
    SIM_SCGC5 |= SIM_SCGC5_PORTB_MASK;
    PORTB_GPCR = 0x00040100; //PtB 2
    GPIOB_PDDR |= (1 << 2); //Setting the bit 2 of the port B as Output
    //use GPIOB_PDOR |= (1UL<<2); to set it
    //use GPIOB_PDOR &= ~(1UL<<2); to clear it

    int len;
    LDD_TDeviceData *SM1_DeviceData;
    SM1_DeviceData = SM1_Init(NULL);

    printf("Hello\n");

    FX1_Init();

    for(;;) {
        // get WHO AM I values
        if (FX1_WhoAmI(&who) != ERR_OK) {
            return ERR_FAILED;
        }
        printf("Who Am I value in decimal \t: %4d\n", who);
        len = sprintf(write, "Who Am I value in decimal \t: %4d\n", who);

        GPIOB_PDOR |= (1UL<<2);
        for(delay = 0; delay < 300000; delay++); //delay
        GPIOB_PDOR &= ~(1UL<<2);
        //
        SM1_SendBlock(SM1_DeviceData, &write, len);

        for(delay = 0; delay < 300000; delay++); //delay
    }
}

```

```

// get raw temperature values
if (FX1_GetTemperature(&temp)!=ERR_OK) {
    return ERR_FAILED;
}
printf("RAW Temperature value in decimal \t: %4d\n",temp);
len = sprintf(write, "RAW Temperature value in decimal \t: %4d\n",temp);
SM1_SendBlock(SM1_DeviceData, &write, len);

for(delay = 0; delay < 300000; delay++); //delay

// Set up registers for accelerometer and magnetometer values
if (FX1_WriteReg8(FX1_CTRL_REG_1, 0x00) != ERR_OK) {
    return ERR_FAILED;
}
if (FX1_WriteReg8(FX1_M_CTRL_REG_1, 0x1F) != ERR_OK) {
    return ERR_FAILED;
}
if (FX1_WriteReg8(FX1_M_CTRL_REG_2, 0x20) != ERR_OK) {
    return ERR_FAILED;
}
if (FX1_WriteReg8(FX1_XYZ_DATA_CFG, 0x00) != ERR_OK) {
    return ERR_FAILED;
}
if (FX1_WriteReg8(FX1_CTRL_REG_1, 0x0D) != ERR_OK) {
    return ERR_FAILED;
}

// Get the X Y Z accelerometer values
accX = FX1_GetX();
accY = FX1_GetY();
accZ = FX1_GetZ();
printf("Accelerometer value \tX: %4d\t Y: %4d\t Z: %4d\n", accX, accY, accZ);
len = sprintf(write, "Accelerometer value \tX: %4d\t Y: %4d\t Z: %4d\n", accX,
accY, accZ);
SM1_SendBlock(SM1_DeviceData, &write, len);

for(delay = 0; delay < 300000; delay++); //delay

// Get the X Y Z magnetometer values
if (FX1_GetMagX(&magX)!=ERR_OK) {
    return ERR_OK;
}
if (FX1_GetMagY(&magY)!=ERR_OK) {
    return ERR_OK;
}
if (FX1_GetMagZ(&magZ)!=ERR_OK) {
    return ERR_OK;
}
printf("Magnetometer value \tX: %4d\t Y: %4d\t Z: %4d\n", magX, magY, magZ);

```

```

        len = sprintf(write, "Magnetometer value \tX: %4d\t Y: %4d\t Z: %4d\n", magX,
magY, magZ);
        SM1_SendBlock(SM1_DeviceData, &write, len);

        for(delay = 0; delay < 300000; delay++); //delay
    }

    /* For example: for(;;) { } */

    /** Don't write any code pass this line, or it will be deleted during code generation. ***/
    /** RTOS startup code. Macro PEX_RTOS_START is defined by the RTOS component.
DON'T MODIFY THIS CODE!!! ***/
    #ifdef PEX_RTOS_START
        PEX_RTOS_START();          /* Startup of the selected RTOS. Macro is defined by the
RTOS component. */
    #endif
    /** End of RTOS startup code. ***/
    /** Processor Expert end of main routine. DON'T MODIFY THIS CODE!!! ***/
    for(;;){}
    /** Processor Expert end of main routine. DON'T WRITE CODE BELOW!!! ***/
} /** End of main routine. DO NOT MODIFY THIS TEXT!!! ***/

/* END main */
/*!
** @}
*/
/*
** #####
**
** This file was created by Processor Expert 10.4 [05.11]
** for the Freescale Kinetis series of microcontrollers.
**
** #####
*/

```

Technical Problems Encountered

Answers to Questions

Q1: Measure the latency of transmitting a string of the “Who Am I” value. (15 points)

a. In the K64F code, include “MK64F12.h” to do this! This will let you use the macros from previous labs. (ex. GPIOB_PDOR)

- b. This can be achieved by setting a single GPIO pin on the K64F high to signify the beginning of transmission. The end of transmission can be marked by setting an Arduino pin HIGH when the Arduino detects the newline character ('\n') in the SPI ISR.
- c. Use Port B pin 2 or 3 on K64F. The beginning of transmission can be marked by setting pin HIGH on the K64F before calling the SM1_SendBlock function.
- d. Configure appropriate clock gating, pin muxing, and PDDR before setting the PDOR. you don't want a bus fault.
- e. In your report:
- Include annotated pictures of the oscilloscope.
 - Give the approximate end-to-end latency of transmitting one string from K64F to Arduino. The end-to-end latency is the difference between the two pulses.

Discuss if the latency measured is reasonable considering the SPI clock rate (Note: SPI transmits one bit at each clock cycle)



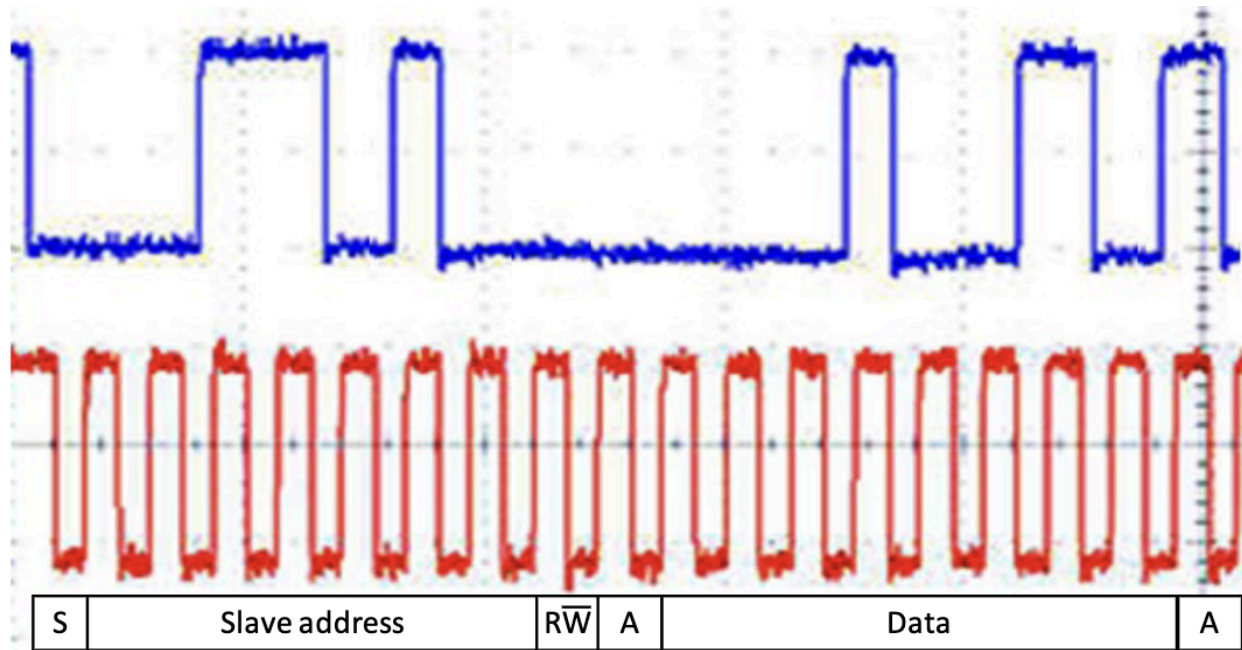
$$\Delta t = 39.30ms - 38.3ms = 1.0ms$$

The latency is reasonable because SPI sends 1 bit per SPI clock cycle.

$$9 \times 8 = 72$$

$$t = \frac{72}{f}$$

Q2: An I2C communication has been captured by a mixed signal oscilloscope. SDA is the blue waveform and SCL is the red one. The address and data portions are indicated in the picture



a) What is the address of the slave?

0x38

b) What is the data value being transferred? Is the master reading or writing the data?

0x8E, the master is writing data to the slave device.

Conclusion

This lab successfully demonstrated SPI serial communication between the FRDM-K64F board and Arduino Uno using Processor Expert.

Contribution

Tina: Report writing, Hardware design

Sangyun: software implementation, testing, Report Writing